Juan Nicolás Pérez Ripe - 202522932
Juan Felipe Moreno Téllez - 202521572
David Ortiz Enciso - 202520816
Tomas Andrés Díaz - 202520393

Punto 1. Probabilidades y utilidad esperada

Tabla

Incendio	Alarma	Cálculo	P(I,A)
1	1	0.10 * 0.90	0.090
1	0	0.10 * 0.10	0.010
0	1	0.90 * 0.05	0.045
0	0	0.90 * 0.95	0.855
		SUMA	1.000

P(A=1) = 0.090 + 0.045 = 0.135

P(I=1 | A=1) = P(A=1 | I=1) * P(I=1) / P(A=1) = 0.090 / 0.135 = 0.667

Implementación de normalize

La función recibe un diccionario de pesos y saca uno nuevo donde cada peso se divide por la suma total. Así conseguimos que las probabilidades sumen 1 y se conservan las claves y orden. Antes de dividir, rechaza con ValueError cuatro casos: diccionario vacío, pesos negativos, pesos no finitos y suma igual a cero. Así ayudando a que el diccionario original no se modifique.

Implementación de expected_utility y best_action

expected_utility calcula EU(a) = Sumatoria (P(s) · U(a, s)): multiplica la probabilidad de cada resultado por su utilidad y suma.

best_action calcula el valor de la utilidad esperada de cada acción también volviendo a usar expected_utility y elige la mayor. La acción escogida sólo cambia cuando aparece una utilidad mayor no vale si es igual , así que en un empate se queda la primera. El diccionario original no se modifica, porque la función crea uno nuevo.

Resultados de sin_evidencia, alarma y sin_alarma, y acción elegida en cada caso

Utilidades usadas, U(acción, I=0) / U(acción, I=1): monitorear 0 / −100, inspeccionar −8 / −25, brigada −30 / −5.

Caso	P(I=1)	EU Monitorear	EU Inspeccionar	EU brigada	Accion
Sin_evidencia	0.100	-10.000	-9.700	-27.500	inspeccionar
alarma	0.667	-66.667	-19.333	-13.333	brigada
sin_alarma	0.012	-1.156	-8.197	-29.711	monitorear

Sin evidencia, el riesgo es moderado y conviene inspeccionar. Con alarma, la probabilidad de incendio sube a 0.667 y la brigada pasa a ser la mejor opción. Sin alarma, la probabilidad baja a 0.012 y basta con monitorear.

Punto 2. Inferencia exacta por enumeración

2a. Implementación de enumeration_ask

Se implementó la función enumeration_ask que permite calcular la probabilidad de una variable teniendo en cuenta la evidencia disponible. Para ello, recorre las variables de la red en orden topológico, multiplica las probabilidades locales y suma los resultados de las posibles combinaciones de las variables ocultas. Finalmente, normaliza los valores obtenidos para que formen una distribución de probabilidad que sume 1.

2b. Verificación con sensor_basico y tabla de bosque (sin_evidencia, alarma, alarma_falla)

Caso	Evidencia	P(incendio=0)	P(incendio=1)
Sin evidencia	Ninguna	0.934840	0.065160
Alarma	Alarma=1	0.670483	0.329517
Alarma y falla	Alarma=1, Falla=1	0.923169	0.076831

Se utilizó enumeration_ask para calcular la probabilidad de incendio teniendo en cuenta la evidencia disponible. El resultado de sensor_basico fue 0.666667, siendo el mismo que el calculado en el punto 1a. La fórmula utilizada es: $P(X \mid e) = \alpha \ \Sigma P(X, e, y)$ Donde X es la variable consultada, e es la evidencia, y representa las variables ocultas y α es el factor de normalización; asimismo, para cada combinación de usa la regla del producto. En el escenario bosque, la probabilidad de incendio

fue de 6.516 % sin evidencia, luego aumentó a 32.952 % al presentarse una alarma; finalmente, disminuyó a 7.683 % cuando también se identificó una falla del sensor. Esto demuestra que la evidencia que se tenga disponible va a modificar el resultado, en este caso la probabilidad de incendio.

2c. Independencia de Humo y Sequedad dado Incendio en cadena_humo: justificación y resultados

En la red cadena_humo, Humo es independiente de Sequedad dado Incendio, porque la probabilidad de Humo depende directamente de Incendio y no de Sequedad; es decir, cuando sabemos que hay incendio hay incendio, saber si existe sequedad no cambia la probabilidad de que haya humo. Esto se verificó mediante las siguientes consultas:

P(Humo=1 | Incendio=1, Sequedad=0) = 0.80

P(Humo=1 | Incendio=1, Sequedad=1) = 0.80

Ambos resultados son iguales, lo que confirma la independencia condicional; sin embargo, esto no significa que Humo y Sequedad sean independientes sin tener en cuenta Incendio, ya que Sequedad influye en la probabilidad de que ocurra un incendio y también influye en la presencia de humo.

Punto 3. Eliminación de variables e inferencia aproximada

3a.Eliminación manual de P(Humo=1) en cadena_humo: factores intermedios y resultado

Datos: P(S=1) = 0.4, P(I=1 | S=0) = 0.02, P(I=1 | S=1) = 0.3, P(H=1 | I=0) = 0.1 y P(H=1 | I=1) = 0.8.

Paso 1: se elimina Sequedad, con f1(I) = Sumatorias de s(P(s) · P(I | s))

Incendio	Cálculo	F1(I)
0	0.6 * 0.98 + 0.4 * 0.70	0.868
1	0.6 * 0.02 + 0.4 * 0.30	0.132

Paso 2: se elimina Incendio, con f2(H) = Sumatoria de i(f1(i) · P(H | i))

Humo	Cálculo	F2(H)

0	0.868 * 0.9 + 0.132 * 0.2	0.8076
1	0.868 * 0.1 + 0.132 * 0.8	0.1924

Resultado: P(Humo=1) = 0.1924

La eliminación de variables del programa da el mismo resultado, con f1(I=0) = 0.868 y f1(I=1) = 0.132.

3b.Enumeración vs eliminación forward y reverse en bosque, y tamaño del mayor factor

Método / orden	Orden de eliminación	P(Incendio=1 | Alarma=1	Mayor factor producto
Enumeración	-	0.3295	-
Eliminación forward	Tormenta, Sequedad, Falla, Humo, Temperatura	0.3295	8 filas
Eliminación reverse	Temperatura, Humo, Falla, Sequedad, Tormenta	0.3295	32 filas

Los tres métodos dan la misma probabilidad. El orden reverse produce un factor más grande porque elimina primero Temperatura, y eso junta a Sequedad, Falla, Incendio y Humo en un solo factor. El orden forward nunca mezcla más de tres variables.

3c. Implementación de prior_sample

La función prior_sample genera una muestra completa de la red; empieza con un diccionario vacío y recorre las variables en orden topológico, de tal manera que cuando llega a una variable sus padres ya tienen un valor. Para cada variable consulta la probabilidad de que sea 1 y la compara con rng.random(); si el número aleatorio es menor, asigna 1 y, en caso contrario, asigna 0. Finalmente retorna el diccionario con todos los valores generados.

3d. Implementación de sampling_ask

La función sampling_ask genera varias muestras usando prior_sample y cuenta cuántas veces la variable consultada vale 0 o 1. Sin rechazo se utilizan todas las muestras y la evidencia debe estar vacía; con rechazo, solo se aceptan las muestras que coinciden completamente con la evidencia. Es

importante que samples representa el número de intentos y no el número de aceptadas; finalmente se normalizan los conteos aceptados y, si no se acepta ninguna muestra, se retorna (None, 0).

3e. Experimentos de muestreo a priori y por rechazo (1000 y 10000 intentos)

Los resultados muestran que al aumentar los intentos normalmente obtenemos una estimación más estable; en muestreo a priori, con 1.000 intentos el error fue 0.009160 y con 10.000 bajó a 0.001760. Con Alarma=1, se aceptaron 191 de 1.000 muestras y 1.841 de 10.000, mientras que con la evidencia rara se aceptaron 0 de 1.000 y solamente 11 de 10.000. Por eso concluimos que, mientras menos probable sea la evidencia, más muestras se rechazan y más difícil es obtener una buena estimación; además, aumentar los intentos ayuda, pero no garantiza que el error siempre sea menor por la aleatoriedad del método.

Método	Evidencia	Intentos	Aceptadas	P estimada Incendio = 1	P exacta	Error
A priori	Sin evidencia	1000	1000	0.056000	0.065160	0.009160
A priori	Sin evidencia	10’000	10’000	0.063400	0.065160	0.001760
Rechazo	Alarma = 1	1000	191	0.277487	0.329517	0.052030
Rechazo	Alarma = 1	10’000	1’841	0.315589	0.329517	0.013928
Rechazo	Rara	1000	0	Sin estimación	0.260778	Sin estimación
Rechazo	Rara	10’000	11	0.090909	0.260778	0.169869

Punto 4. MDP e iteración de valores

4a. Estados, acciones, transiciones, recompensas, terminales y descuento de cuadrícula

En cuadrícula, cada estado representa la posición actual del dron; las acciones disponibles son N, E, S y W y las transiciones dependen de la acción elegida y del ruido de 0.20, por lo cual el movimiento deseado ocurre con probabilidad 0.80 y cada posible desviación lateral con 0.10. La recompensa normal por movimiento es -0.04; G es el estado terminal favorable con recompensa +1 y X es el terminal desfavorable con recompensa -1, asimismo el descuento utilizado es 0.90; la posición actual y la acción elegida son suficientes para determinar las probabilidades del siguiente estado, ya

que no importa el camino que tomó el dron anteriormente; de esta manera, dos drones ubicados en la misma posición y que realicen la misma acción tienen las mismas probabilidades de transición.

4b. Implementación de action_value

La función action_value permite calcular el valor esperado de realizar una acción desde un estado determinado. Para esto, tiene en cuenta las posibles posiciones a las que puede llegar el dron, la probabilidad de cada movimiento, la recompensa recibida y el valor futuro de cada estado. Para esto se utiliza la ecuación de Bellman, la cual multiplica la recompensa inmediata más el valor futuro descontado por la probabilidad de cada posible movimiento. Finalmente, suma todos los resultados para obtener el valor esperado de ejecutar esa acción.

4c. Implementación de value_iteration

La función value_iteration permite calcular el valor óptimo de cada estado mediante un proceso repetitivo de barridos. Inicialmente, todos los estados tienen un valor de cero. En cada barrido se calcula un nuevo valor para cada estado, seleccionando la acción que proporciona el mayor valor esperado. Para realizar las actualizaciones se utilizan únicamente los valores del barrido anterior, evitando mezclar valores nuevos y antiguos. Después de cada barrido se calcula el cambio máximo entre los valores anteriores y los nuevos. Este proceso continúa hasta que el cambio sea menor que la tolerancia establecida o se alcance el máximo de iteraciones permitido. Los estados terminales mantienen un valor de cero, ya que la recompensa correspondiente se recibe al entrar en ellos y no se vuelve a recibir posteriormente.

4d. Verificación con control y propagación de valores en cuadrícula

Control

Se ejecutó el escenario control, utilizando los parámetros predeterminados:

- Ruido: 0.

- Recompensa por movimiento: -0.04.

- Descuento: 0.9

Los resultados obtenidos fueron:

Indicador	Resultado
Barridos	2
Convergencia	Si
Valor del estado inicial	1

Acción óptima	E

Valores de las acciones en el estado inicial:

Acción	Valor Esperado
Norte (N)	0.86
Este (E)	1
Sur (S)	0.86
Oeste (O)	0.86

El resultado demuestra que la mejor decisión es moverse hacia el este, ya que permite llegar directamente al punto de descarga y recibir la recompensa terminal de +1. Las demás acciones mantienen al dron en su posición inicial debido a los bordes, por lo que reciben una recompensa de -0.04.

Cuadrícula

Se ejecutó el escenario cuadrícula, utilizando los siguientes parámetros:

- Ruido: 0.2

- Recompensa por movimiento: -0.04.

- Descuento: 0.9

- Tolerancia: 10^-8

Resultados de los primeros y últimos barridos:

Barrido	Cambio Máximo	Valor Inicial
0	-	-
1	0.792	-0.04
2	0.56304	-0.076
3	0.398909	-0.1084
24	3.72829e-08	0.373852

25	1.57695e-08	0.373852
26	6.91094e-09	0.373852

Al principio, los valores cambian considerablemente porque el algoritmo todavía está incorporando la información de las recompensas futuras. A medida que avanzan los barridos, los cambios son cada vez menores, hasta que los valores se estabilizan. Finalmente, el algoritmo convergió después de 26 barridos, obteniendo un valor de 0.373852 para el estado inicial.

Punto 5. Extracción y análisis de políticas

5a. Implementación de extract_policy

La función extract_policy utiliza los valores calculados anteriormente mediante value_iteration para determinar cuál es la mejor acción que debe realizar el dron en cada estado. Para esto, evalúa las acciones disponibles utilizando action_value y selecciona aquella que tenga el mayor valor esperado. Si varias acciones tienen el mismo valor, conserva la primera según el orden establecido: norte, este, sur y oeste. En los estados terminales no se selecciona ninguna acción, por lo que se asigna el valor None. De esta manera, se obtiene una política que indica qué acción conviene realizar en cada posición, teniendo en cuenta las posibles transiciones, las recompensas y el ruido del entorno.

5b. Atajo con ruido 0 vs 0.3

Se ejecutó el escenario atajo, modificando únicamente el ruido y manteniendo los demás parámetros iguales:

- Recompensa por movimiento: -0.04.

- Descuento: 0.95

Indicador	Ruido 0	Ruido 0.3
Valor Estado Inicial	1.366587	0.711171
Acción Inicial	E	N
Barridos	9	93
Convergencia	Si	Si

Con ruido 0, el dron elige moverse hacia el este, ya que puede avanzar directamente hacia el punto de descarga sin desviaciones. Cuando el ruido aumenta a 0.3, la acción inicial cambia al norte. Esto ocurre porque ahora existe una mayor probabilidad de que el viento desvíe al dron, por lo que resulta más conveniente tomar una dirección que reduzca el riesgo de terminar en una zona desfavorable. También se observa que el valor del estado inicial disminuye de 1.366587 a 0.711171. Esto demuestra que un mayor nivel de incertidumbre puede reducir el beneficio esperado de la misión y modificar la política óptima.

5c. zona_riesgo variando living_reward y discount

Cambio de la recompensa por movimiento

Se mantuvieron constantes el ruido en 0.10 y el descuento en 0.95. Únicamente se modificó la recompensa por movimiento.

Indicador	Living reward -0.04	Living reward -0.3
Valor Estado Inicial	1.003639	-0.715505
Acción Inicial	S	E
Barridos	28	27
Convergencia	Si	Si

Cuando la recompensa por movimiento cambia de -0.04 a -0.30, desplazarse resulta más costoso para el dron. Por esta razón, el valor del estado inicial disminuye considerablemente y pasa a ser negativo. También cambia la acción inicial de sur a este, mostrando que el dron modifica su decisión para buscar una alternativa que permita reducir los costos asociados a los movimientos.

Cambio del factor de descuento

Se mantuvieron constantes el ruido en 0.10 y la recompensa por movimiento en -0.04. Únicamente se modificó el descuento.

Indicador	Discount 0.5	Discount 0.95
Valor Estado Inicial	-0.073433	1.003639
Acción Inicial	N	S
Barridos	19	28

Convergencia	Si	Si

Cuando el descuento es 0.50, las recompensas futuras pierden importancia rápidamente, por lo que el algoritmo prioriza más los resultados inmediatos. En cambio, cuando el descuento aumenta a 0.95, las recompensas futuras tienen mayor importancia en el cálculo. Se observa que el valor del estado inicial cambia de -0.073433 a 1.003639 y que la acción inicial pasa de norte a sur. Esto demuestra que el factor de descuento puede modificar tanto los valores de los estados como las decisiones que conforman la política.

Punto 6. Reflexión

6a. Análisis de los algoritmos y resultados

- **a) Evidencia, probabilidad y utilidad esperada**

La evidencia cambia la probabilidad de incendio. Sin evidencia es 0.10. Al observar la alarma sube a 0.667, porque es mucho más probable que suene con incendio (0.90) que sin él (0.05). Si la alarma no suena, baja a 0.012. La probabilidad mide qué tan probable es el incendio. La utilidad esperada combina esa probabilidad con las consecuencias de cada acción. Por eso la decisión no depende solo de qué tan probable sea el incendio. Con apenas 0.10 ya conviene inspeccionar, porque no actuar ante un incendio cuesta -100. Con 0.667 la brigada es la mejor opción, aunque cueste -30 si no hay incendios. Con 0.012 lo mejor es monitorear, porque el riesgo es casi nulo.

- b) Enumeración vs eliminación y orden de eliminación
- c) Intentos, evidencia, error y muestras aceptadas

La cantidad de intentos influye en la cantidad de muestras disponibles para realizar la estimación; sin evidencia todas las muestras son aceptadas, mientras que con muestreo por rechazo solamente se utilizan las que coinciden con la evidencia. Con Alarma=1 se aceptaron 191 de 1.000 intentos y 1.841 de 10.000; sin embargo, con la evidencia rara se aceptaron 0 de 1.000 y solamente 11 de 10.000. En ese sentido, mientras menos probable sea la evidencia, más muestras se descartan y mayor puede ser el error; asimismo, aumentar los intentos normalmente permite obtener más muestras útiles y una aproximación más estable, pero no garantiza que el error siempre disminuya debido al carácter aleatorio del método.

- **b) Enumeración y eliminación de variables**

La enumeración y la eliminación de variables deben producir el mismo resultado porque ambos algoritmos calculan probabilidades exactas utilizando la misma red bayesiana y la misma evidencia. Al comparar los órdenes de eliminación, se obtuvo una probabilidad de incendio de 0.329517 en ambos casos; sin embargo, el orden forward generó un factor máximo de 8 filas, mientras que

reverse generó uno de 32 filas. Esto demuestra que cambiar el orden de eliminación puede aumentar o disminuir el tamaño de los factores intermedios y el costo.

- **e) Ruido, recompensa y descuento, y política vs ruta fija**

Los experimentos muestran que el ruido, la recompensa por movimiento y el factor de descuento pueden modificar los valores de los estados y las decisiones del dron. En el escenario atajo, al aumentar el ruido de 0 a 0.3, la acción inicial cambió de este a norte y el valor del estado inicial disminuyó de 1.366587 a 0.711171. Esto demuestra que una mayor incertidumbre puede hacer que una acción diferente resulte más conveniente.

En zona_riesgo, al cambiar la recompensa por movimiento de -0.04 a -0.30, el valor inicial disminuyó de 1.003639 a -0.715505 y la acción cambió de sur a este. Por otro lado, al modificar el descuento de 0.50 a 0.95, el valor inicial pasó de -0.073433 a 1.003639 y la acción cambió de norte a sur. Finalmente, una política no es lo mismo que una ruta fija, ya que esta indica qué acción debe realizarse en cada estado, mientras que una ruta fija representa una secuencia específica de movimientos. La política es más flexible porque considera las posibles situaciones en las que puede encontrarse el dron y permite tomar decisiones según el estado actual.

- **d) Barridos de valores y criterio de parada**

Los barridos muestran cómo los valores de los estados se actualizan progresivamente hasta encontrar una solución estable. Al principio, los valores cambian considerablemente porque el algoritmo comienza desde cero y va incorporando las recompensas futuras. Después, los cambios disminuyen hasta que los valores prácticamente dejan de variar. Para detener el proceso se utilizó un criterio de convergencia basado en el cambio máximo entre dos barridos consecutivos. El algoritmo termina cuando este cambio es inferior a 10^-8 o cuando alcanza el máximo de 1000 iteraciones.

6b. Reflexión sobre el uso de IA

Durante el desarrollo del taller se utilizó inteligencia artificial como herramienta de apoyo para comprender los algoritmos y solucionar errores. Cada integrante desarrolló una versión propia de las funciones y luego utilizó la IA para recibir sugerencias sobre su código. Además, las implementaciones se verificaron mediante los escenarios establecidos.